

COLLEGE FOOTBALL FANTASY

Authentication, Authorization & Security Requirements

Security Foundation Document

Project	College Football Fantasy — Team 4
Sprint	Sprint 1 — Foundation
Owner Function	Cybersecurity
Product Manager	Aliyu Muhammed
Document Version	1.0
Date	September 2026
Status	Draft for Team Review

Document Control

Version	Date	Status	Owner
1.0	September 2026	Draft for Team Review	Cybersecurity

Approval / Review

This document is intended for review by the Product Manager, engineering team, and relevant project stakeholders before implementation.

Role	Name	Approval / Date
Cybersecurity	Team 4 Cybersecurity	Pending
Product Manager	Aliyu Muhammed	Pending
Engineering Lead	To be assigned	Pending

Table of Contents

1. Purpose
2. Scope
3. Security Objectives
4. Authentication Requirements
5. Authorization Requirements
6. Role-Based Access Control (RBAC)
7. API Security Requirements
8. Database Security Requirements
9. Secure Handling of Player, Team & Game Data
10. Scoring Integrity & Business-Logic Security
11. Secrets Management
12. Logging, Monitoring & Audit
13. Frontend–Backend Security Boundary
14. Third-Party API Security
15. Security Testing Requirements
16. Threats Addressed
17. Security Acceptance Criteria
18. Implementation Priorities
19. Roles & Responsibilities
20. Conclusion

1. Purpose

This document defines the authentication, authorization, access-control, API, data-protection, logging, monitoring, and related security requirements for the College Football Fantasy platform.

The purpose is to establish a secure foundation for the platform before production implementation. The requirements are intended to guide product, frontend, backend, database, cloud, and cybersecurity work throughout development and testing.

2. Scope

These requirements apply to the web/mobile client, backend REST APIs, authentication services, PostgreSQL database, fantasy-team and league functionality, scoring/ranking services, administrative functions, third-party College Football Data API integration, cloud infrastructure, logs, secrets, and security testing.

The requirements cover user identity, permissions, protected resources, application interfaces, sensitive configuration, security events, and the integrity of fantasy scoring and rankings.

3. Security Objectives

The platform shall:

- Ensure that only authenticated users can access protected resources.
- Ensure that authenticated users can perform only actions permitted by their role and resource ownership.
- Protect credentials, tokens, secrets, and personal/account information.
- Prevent unauthorized modification of fantasy teams, leagues, scores, rankings, and other game-critical data.
- Protect APIs against common abuse and application-layer attacks.
- Provide sufficient security logging and monitoring to detect and investigate suspicious activity.
- Apply least privilege and secure-by-default principles across application and infrastructure components.
- Preserve confidentiality, integrity, and availability of platform data and services.

4. Authentication Requirements

4.1 User registration and login

- The system shall require authentication before access to protected resources.
- User registration shall validate required fields and reject invalid or malformed input.
- Passwords shall never be stored in plaintext.
- Passwords shall be stored using a strong, modern password-hashing algorithm with an appropriate work factor.
- Authentication endpoints shall use HTTPS/TLS.
- The system shall provide clear but non-revealing authentication errors so that responses do not unnecessarily disclose whether an account exists.

4.2 Password security

- Passwords shall meet defined minimum length and complexity requirements appropriate to the product.
- Password reset shall use short-lived, single-use reset tokens.
- Password-reset tokens shall not be stored or logged in plaintext where avoidable.
- Password changes shall invalidate relevant existing sessions/tokens according to the application's session policy.

4.3 Session/token security

- Access tokens shall be short-lived where practical, with controlled refresh mechanisms.

- Tokens shall be validated for signature, issuer, audience, expiry, and other required claims.
- Sensitive tokens shall not be exposed in URLs or application logs.
- Logout/session revocation shall be supported according to the selected authentication architecture.
- The system shall prevent reuse of expired or invalid tokens.

4.4 Brute-force and automated-abuse protection

- Login and password-reset endpoints shall be rate limited.
- Repeated failed authentication attempts shall trigger appropriate throttling or temporary protective controls.
- Security controls shall avoid creating an easy denial-of-service mechanism against legitimate users.

5. Authorization Requirements

- Authorization shall be enforced server-side; frontend controls shall not be treated as security boundaries.
- Every protected API endpoint shall define the required permission or role.
- A user shall be permitted to access another user's resources only where an explicit business rule allows it.
- Resource ownership shall be checked on the backend for user-owned objects such as profiles and fantasy teams.
- Administrative actions shall require elevated permissions.
- Authorization failures shall return appropriate responses without revealing sensitive implementation details.
- Privilege checks shall occur before sensitive operations are executed.
- Default access shall be deny-by-default unless a permission is explicitly granted.

6. Role-Based Access Control (RBAC)

The initial RBAC model shall use least privilege and may be refined as product requirements evolve.

Player/User:

- Manage own account/profile.
- Create, edit, and manage permitted fantasy teams.
- Join and participate in permitted leagues.
- View authorized player, team, gameweek, score, and leaderboard information.
- Cannot perform administrative operations or modify system-wide scoring rules.

League Manager:

- Perform approved management actions for leagues they own/manage.
- Manage permitted league settings and membership functions.
- Cannot access unrelated users' private information or system administration functions unless separately authorized.

Admin:

- Manage users, roles, leagues, platform configuration, and approved administrative operations.
- Access security/audit information according to operational need.
- Administrative privileges shall be separated from ordinary user privileges.

Developer/Service Account:

- Used only for approved application or operational functions.
- Permissions shall be narrowly scoped to required services/resources.

- Human users shall not share service-account credentials.

RBAC implementation shall support permission checks at the API/service layer. Role names alone shall not replace resource-level authorization checks where ownership or context matters.

7. API Security Requirements

- All production API traffic shall use HTTPS/TLS.
- Protected endpoints shall validate authentication before processing protected requests.
- API endpoints shall enforce authorization and resource ownership checks.
- Request payloads shall be validated against expected types, ranges, formats, and business rules.
- Server-side validation shall be performed even when client-side validation exists.
- APIs shall implement rate limiting appropriate to endpoint sensitivity and expected traffic.
- CORS shall be restricted to approved origins and configured deliberately.
- Sensitive data shall not be returned unless required by the requesting user's authorized function.
- API errors shall not expose stack traces, database errors, secrets, tokens, or internal infrastructure details.
- HTTP methods shall be restricted to the operations supported by each endpoint.
- The API shall protect against injection, broken access control, authentication failures, and other common web/API vulnerabilities.
- Security-sensitive endpoints shall generate auditable security events.
- Third-party API credentials shall be stored as secrets and never embedded in frontend code.

8. Database Security Requirements

- PostgreSQL access shall use dedicated credentials with least-privilege permissions.
- Application services shall not use database superuser accounts for normal operations.
- Database credentials shall be stored in an approved secrets-management mechanism.
- SQL queries shall use parameterization or a secure ORM/query mechanism.
- Sensitive database access shall be restricted to authorized services and administrators.
- Backups shall be protected from unauthorized access and periodically tested for recoverability.
- Database connections shall use encryption where supported by the deployment architecture.
- Production database access shall be separated from development/test access.
- Database schema changes shall be controlled and reviewed.
- Security-relevant database operations shall be logged where appropriate.

9. Secure Handling of Player, Team & Game Data

The platform shall distinguish between public sports data and user/account data.

- Player and college-team information obtained from approved external sources shall be validated before use.
- User-created fantasy-team data shall be protected by authentication and authorization controls.
- Users shall not be able to alter authoritative player statistics or game results unless an approved administrative workflow explicitly permits it.
- Gameweek scores, rankings, and scoring calculations shall be generated by trusted backend services rather than relying on client-submitted values.

- The server shall validate all scoring-related inputs.
- Critical game data shall have controlled update paths and appropriate audit records.
- Data returned to clients shall follow the principle of data minimization.
- Personal/account information shall not be exposed through public leaderboard or player-data endpoints unless explicitly required.

10. Scoring Integrity & Business-Logic Security

Because fantasy scoring and rankings are core game functions, the backend shall be treated as the authoritative source for scoring.

- Clients shall not be trusted to submit final scores, rankings, or other authoritative game results.
- Scoring calculations shall be performed or verified by trusted backend services.
- Inputs used in scoring shall be validated for type, range, completeness, and expected source.
- Duplicate processing of the same game/stat update shall be prevented where necessary through idempotency or equivalent controls.
- Changes to scoring rules shall require controlled authorization and review.
- Security-relevant scoring or ranking changes shall be auditable.
- The system shall detect or reject attempts to manipulate game-critical values through direct API requests.

11. Secrets Management

- Secrets shall never be committed to Git repositories.
- API keys, database passwords, signing keys, refresh secrets, and similar credentials shall be stored in environment-specific secret/configuration management.
- Secrets shall not be hard-coded in frontend applications.
- Production and development secrets shall be separated.
- Access to secrets shall follow least privilege.
- Secrets shall be rotated when compromise is suspected and according to an established operational policy.
- Logs shall be reviewed to ensure secrets are not accidentally recorded.
- Source-code repositories shall be scanned for accidentally committed secrets where practical.

12. Logging, Monitoring & Audit

The platform shall generate security-relevant events sufficient for detection, investigation, and accountability.

Events to consider include:

- Successful and failed authentication attempts.
- Password changes and password-reset requests.
- Account lockouts or significant rate-limit events.
- Role/permission changes.
- Administrative actions.
- Creation, modification, and deletion of security-sensitive resources.
- Suspicious API activity and repeated authorization failures.
- Changes affecting scoring, rankings, or other game integrity controls.

- Important service or security configuration changes.

Logs shall:

- Avoid storing passwords, authentication secrets, or unnecessary personal information.
- Include sufficient timestamp/context information for investigation.
- Be protected against unauthorized modification or deletion.
- Be monitored for meaningful security alerts.
- Have an appropriate retention policy based on operational and project requirements.

13. Frontend–Backend Security Boundary

The frontend is an untrusted client from a security perspective.

- UI restrictions shall improve usability but shall not be relied upon for authorization.
- All security decisions shall be enforced by the backend.
- The frontend shall not contain privileged secrets.
- The frontend shall not determine authoritative scores or permissions.
- Tokens and sensitive client-side data shall be handled using the security mechanisms appropriate to the chosen application architecture.
- Security headers and browser protections shall be configured where applicable.

14. Third-Party API Security

The College Football Data API integration shall be treated as an external trust boundary.

- API credentials shall be stored securely and kept server-side where required.
- External data shall be validated before entering trusted application workflows.
- External API failures, unexpected responses, and rate limits shall be handled safely.
- The application shall not blindly trust external input.
- The system shall monitor usage and protect credentials from exposure.
- External data shall be mapped into controlled internal models before being consumed by critical business logic.

15. Security Testing Requirements

Before release, the team should perform security validation appropriate to the implementation stage, including:

- Authentication and authorization testing.
- RBAC and privilege-escalation testing.
- Resource-ownership/IDOR-style testing.
- API input-validation testing.
- Rate-limit and abuse-control testing.
- Injection testing.
- Session/token security testing.
- Secrets and configuration review.
- Dependency and vulnerability scanning.
- Security logging and alert verification.

- Scoring-integrity and business-logic abuse testing.

Critical findings shall be tracked, prioritized, remediated, and retested before production release where feasible.

16. Threats Addressed

The requirements are designed to reduce risks including:

- Credential theft and account takeover.
- Brute-force and credential-stuffing attempts.
- Broken access control and privilege escalation.
- Unauthorized access to another user's fantasy team or account data.
- API abuse and automated attacks.
- Injection and malformed-input attacks.
- Exposure of API keys, database credentials, or signing secrets.
- Unauthorized modification of scores, rankings, or game data.
- Insider misuse of excessive privileges.
- Insufficient logging that prevents detection or investigation.
- Compromise of third-party API integration paths.

17. Security Acceptance Criteria

The Sprint 1 foundation shall be considered security-ready for the next implementation stage when the team can demonstrate that:

- AC-01: Protected resources require valid authentication.**
- AC-02: Expired, invalid, or tampered authentication tokens are rejected.**
- AC-03: Authorization is enforced on the backend for protected endpoints.**
- AC-04: Users cannot access or modify another user's protected resources without explicit permission.**
- AC-05: Administrative functions require appropriate privileges.**
- AC-06: Passwords are not stored in plaintext.**
- AC-07: Authentication and sensitive API endpoints have appropriate rate limiting.**
- AC-08: API input is validated server-side.**
- AC-09: Production secrets are not hard-coded or committed to source control.**
- AC-10: Database access follows least privilege.**
- AC-11: Security-relevant authentication, authorization, administrative, and integrity events are logged.**
- AC-12: Logs do not expose passwords, tokens, or other secrets.**
- AC-13: Client-side controls cannot bypass backend authorization.**
- AC-14: Scoring and ranking values cannot be trusted directly from the client.**
- AC-15: Identified high-severity security findings are addressed or formally accepted before production release.**

18. Implementation Priorities

Priority 1 — Foundation:

- Authentication architecture.

- Backend authorization middleware.
- Initial RBAC model.
- Secure token/session design.
- Secrets-management approach.
- API input validation and rate limiting.

Priority 2 — Data & Integrity:

- Database least privilege.
- Resource ownership checks.
- Scoring/ranking integrity controls.
- Audit/security logging.

Priority 3 — Hardening:

- Security testing and vulnerability scanning.
- Monitoring and alerting.
- Dependency/configuration hardening.
- Incident-response preparation.

19. Roles & Responsibilities

Cybersecurity:

- Define security requirements and security controls.
- Review authentication, authorization, RBAC, API, database, secrets, logging, and integrity controls.
- Support threat modelling and security testing.
- Track security risks and remediation.

Backend Engineering:

- Implement server-side authentication, authorization, validation, secure data access, and business-logic protections.

Frontend Engineering:

- Implement secure user flows and UI controls without treating frontend restrictions as authorization.

DevOps/Cloud:

- Secure infrastructure, secrets, networking, deployments, monitoring, and environment separation.

Product Management:

- Confirm business rules, roles, permissions, and acceptance criteria.
- Prioritize security requirements alongside product requirements.

20. Conclusion

Authentication and authorization form the security foundation of the College Football Fantasy platform. The application must follow least privilege, deny-by-default authorization, secure credential handling, server-side validation, protected APIs, secure database access, secrets management, security logging, and game-integrity controls.

These requirements provide the baseline for Sprint 1 and should be reviewed with the Product Manager and engineering team before implementation. Any deviation from a requirement should be documented with its reason, risk, compensating control, and approval.